

Symbiosis Artificial Intelligence Institute

MINI PROJECT REPORT

End-Semester Examination (Lab ESE) - October 2026 | Course: Cryptography

PQFS 2.0

Post-Quantum File Sharing System

Multi-family hybrid KEM, Shamir multipath key sharing, crypto-agility, shelf-life policy engine, transparency logs and metadata privacy

Submitted by

Pushkar Wagh (PRN: 25030421030)

Vivaan Kriplani (PRN: 25030421049)

Under the guidance of

Dr. Reema Roychaudhary

Academic Year 2026-27 | Submission date: 24 October 2026

Declaration

We hereby declare that this mini project report titled "PQFS 2.0 - Post-Quantum File Sharing System" is the record of work carried out by us. The cryptographic building blocks (AES-GCM, X25519, Ed25519, scrypt, HKDF from the Python 'cryptography' library; ML-KEM, HQC, ML-DSA and SLH-DSA from the Open Quantum Safe 'liboqs' library) are used as external libraries. The protocol design, the Shamir secret-sharing, Padme padding, Merkle transparency log, policy engine, relay server, command-line client and test suite were implemented by us in a single Python program (about 2,200 lines). All results reported here were produced by running that program.

Pushkar Wagh (PRN 25030421030): _____ Vivaan Kriplani (PRN 25030421049): _____ Date: 24 October 2026

Abstract

Encrypted files that are stolen today can be stored and decrypted later when a large quantum computer exists (the "harvest-now, decrypt-later" threat). Classic file-sharing tools protect files with elliptic-curve or RSA key exchange, which Shor's algorithm would break. PQFS 2.0 is a command-line file-sharing system that is designed so that a file stays confidential even if one cryptographic family is broken.

The file is encrypted once, in 64 KiB authenticated chunks, with a random 256-bit data key (AES-256-GCM). That key is split with Shamir secret sharing over GF(256). Every share is wrapped for the recipient using a hybrid key-encapsulation combiner that mixes X25519 (elliptic curve), ML-KEM (lattice, FIPS 203) and HQC (code-based) shared secrets through one HKDF, so an attacker must break all families. Shares are spread over independent relay servers: fewer than t relays learn nothing about the key, even against an adversary with unlimited computing power. A shelf-life policy engine converts "how many years must this stay secret" and the apparent sensitivity of the file into one of three security tiers. When an algorithm is later declared weak, an `audit` command finds affected files and `rewrap` re-issues only the small key-wrap layer, leaving the ciphertext untouched. Relays keep RFC 6962-style Merkle logs signed with the hash-based SLH-DSA, giving users receipts and the ability to detect history rewriting. File names, sizes, hashes and the sender identity are kept inside an encrypted trailer, sizes are hidden with Padme padding and recipients are never named in the package.

The prototype passed all 24 unit self-tests and a 12-scenario end-to-end demonstration that included tamper detection, rollback and downgrade detection, weak-topology refusal, 2-of-3 threshold approval, algorithm deprecation with automatic rewrap, and log consistency monitoring. Measured on a single cloud CPU core, the symmetric layer reached about 186 MiB/s encryption and 150 MiB/s verified decryption on a 50 MiB file; the heaviest tier (T3) adds about 23.1 ms to wrap and 35.3 ms to unwrap one key share. The system is an educational prototype and has not been security audited.

Keywords: post-quantum cryptography, hybrid KEM, ML-KEM, HQC, Shamir secret sharing, crypto-agility, transparency log, Merkle tree, SLH-DSA, metadata privacy.

Table of Contents

Declaration 2
Abstract 3
1. Introduction 5
2. Problem Statement and Objectives 6
3. Background and Related Work 7
4. System Design 8
4.1 Architecture 8
4.2 Key hierarchy 8
4.3 Hybrid KEM combiner 8
4.4 Security tiers and the policy engine 9
4.5 Package format (metadata privacy) 9
4.6 Protocols 9
4.7 Relay channel and transparency log 10
4.8 Topology rule 10
5. Implementation 11
6. Testing, Results and Analysis 13
6.1 Test environment 13
6.2 Unit tests (selftest) 13
6.3 End-to-end scenarios (demo) 13
6.4 Cryptographic primitive performance 14
6.5 Cost of each security tier 16
6.6 File encryption throughput and storage overhead 16
6.7 Padme padding 17
6.8 Secret sharing and transparency log cost 18
6.9 Summary of findings 18
7. Security Analysis and Limitations 19
8. Conclusion and Future Work 20
References 21
Appendix A: How to run the prototype 22
Appendix B: Mapping to evaluation rubrics 23

1. Introduction

1.1 Background

Most secure file transfer today relies on public-key algorithms such as RSA and elliptic-curve Diffie-Hellman to agree on session keys. Shor's quantum algorithm would solve the underlying mathematical problems efficiently, so a sufficiently large quantum computer would break every recorded session that used them. Symmetric ciphers such as AES-256 are not broken by known quantum attacks (Grover's algorithm only halves the effective key length), which is why the weak point is the key exchange, not the bulk encryption.

Because intelligence agencies and criminals can copy ciphertext now and wait, data that must stay secret for 10-30 years (medical records, legal documents, private keys, trade secrets) is already at risk. NIST has therefore standardised post-quantum algorithms: ML-KEM (FIPS 203) for key encapsulation, ML-DSA (FIPS 204) and SLH-DSA (FIPS 205) for signatures, and has selected HQC as a code-based backup key-encapsulation mechanism.

However, these algorithms are young. Several earlier candidates (for example SIKE and Rainbow) were broken by classical attacks late in the NIST process. A responsible design therefore must not bet everything on one new algorithm.

1.2 Motivation

Existing hobby and student projects typically stop at "AES + password over TCP" (this was version 1 of our own project). We wanted a system that treats the quantum threat as an engineering problem with several layers: what if one math family fails, what if some servers are hostile, what if an algorithm is retired in five years, and what can the server learn about the file.

1.3 Scope of this report

Chapter 2 states the problem and objectives. Chapter 3 summarises related standards and techniques. Chapter 4 presents the design and Chapter 5 the implementation. Chapter 6 reports test results and performance measurements with analysis. Chapter 7 discusses security properties and limitations, and Chapter 8 concludes.

2. Problem Statement and Objectives

2.1 Problem statement

Design and implement a file-sharing system whose confidentiality, integrity and metadata privacy do not depend on any single cryptographic assumption or on any single server, that can be upgraded when an algorithm is deprecated without re-encrypting the data, and whose servers can be held accountable for what they do.

2.2 Threat model

Adversary	Capability	Design response
Harvest-now-decrypt-later	Records all traffic and stored data, later owns a quantum computer	Hybrid KEM (needs all families broken); Shamir sharing (information-theoretic); PQ-hybrid relay channel
Cryptanalyst	Breaks one algorithm family (e.g. a lattice attack appears)	Multi-family combiner; crypto-agility with rewrap
Malicious relay (minority)	Reads stored data, tampers, replays old key-wraps, rewrites its log	Shares spread over relays; AEAD per chunk; signed manifest; epoch rollback check; Merkle log + SLH-DSA
Network attacker	Man-in-the-middle on relay connections	Server authenticated by hash-based signature; key pinning (TOFU or ticket fingerprint)
Curious operator	Inspects stored packages for names, sizes, senders	Encrypted trailer, Padme padding, equal chunks, anonymous clients, no recipient identifiers

2.3 Objectives

- **O1:** Hybrid key encapsulation across at least three independent mathematical families.
- **O2:** Split the file key over several relays with a threshold t-of-n so that fewer than t relays learn nothing.
- **O3:** Crypto-agility: replace the key-wrap layer without touching ciphertext, and resist downgrade and rollback attacks.
- **O4:** A transparent, explainable policy engine that chooses security parameters from the required secrecy lifetime and file sensitivity.
- **O5:** Threshold custody (several people must cooperate to open a file) and verifiable, append-only relay logs.
- **O6:** Metadata privacy: hide file name, exact size, sender identity and recipient identity from relays.
- **O7:** A complete working prototype with automated tests, a one-command demo and measured performance.

2.4 Out of scope

Hiding IP addresses and timing (a mix-network such as Tor would be needed), a graphical interface, user account management, and a formal security proof or third-party audit.

3. Background and Related Work

Technique	Role in PQFS 2.0	Reference
ML-KEM (Kyber)	Lattice-based KEM; fast and compact; main post-quantum layer	NIST FIPS 203 (2024)
HQC	Code-based KEM; independent math family as insurance against a lattice break	NIST selection (2025); liboqs
X25519	Elliptic-curve DH; classical layer, protects if PQ code has an implementation bug	RFC 7748
ML-DSA / Ed25519	Hybrid signature on manifests and releases (both must verify)	FIPS 204; RFC 8032
SLH-DSA (SPHINCS+)	Hash-based signature for relay identity and log tree heads; security relies only on hash functions	NIST FIPS 205
AES-256-GCM	Authenticated encryption of file chunks, trailer, key wraps and channel records	NIST SP 800-38D
Shamir secret sharing	Threshold splitting of the data key over GF(256); information-theoretic	Shamir, CACM 1979
Merkle transparency log	Append-only log with inclusion and consistency proofs	RFC 6962 / RFC 9162
Padme padding	Hides file size with at most about 12% overhead	Nikitin et al., PETS 2019 (PURBs)
HKDF, scrypt	Key derivation; passphrase protection of identity files	RFC 5869; RFC 7914

The hybrid combiner follows the principle used by hybrid key-exchange drafts for TLS: derive the final key from all component shared secrets and bind in the ciphertexts and public keys, so the result is secure as long as at least one component is secure. Our contribution is combining this with threshold sharing across servers, an upgrade (rewrap) workflow, and logging in one coherent tool.

4. System Design

4.1 Architecture

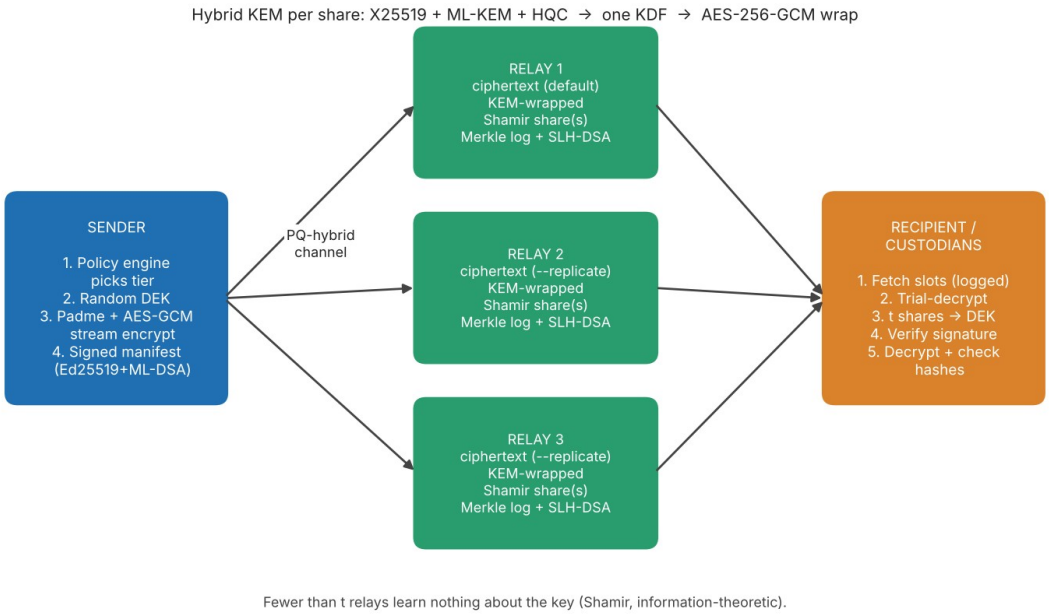


Figure 4.1: PQFS 2.0 architecture. Ciphertext and key shares travel to independent relays; the recipient (or t custodians) reassembles the key.

There are three roles. The sender encrypts the file and publishes the ciphertext and the wrapped key shares. Relays are untrusted store-and-forward servers that also run a transparency log. The recipient (or a group of custodians) fetches the key shares, rebuilds the data key, verifies the sender and decrypts. A small JSON ticket (relay addresses, relay key fingerprints, package id) is the only thing sent out-of-band; it contains no secrets.

4.2 Key hierarchy

- **DEK (data encryption key):** 256 random bits per file. Separate sub-keys for file chunks, the metadata trailer and the descriptor MAC are derived from it with HKDF.
- **Shares:** the DEK is Shamir-split into n shares, any t reconstruct it, t-1 give no information.
- **Slot:** one share wrapped for one holder with AES-256-GCM under a key derived from the hybrid KEM combiner (X25519 + ML-KEM + HQC).
- **Slot set / epoch:** a versioned collection of slots with a descriptor (tier, suites, n, t, epoch). Rewrap creates epoch 2, 3, … for the same DEK.

4.3 Hybrid KEM combiner

For each share the sender runs one encapsulation per algorithm in the selected suite and feeds all shared secrets into a single HKDF call. The HKDF 'info' input contains the hash of the context (which includes the descriptor hash, hence the epoch and the suite list) and a hash of every (algorithm name, public key, ciphertext) triple. Consequently the wrap key changes if any ciphertext, public key, suite or epoch is altered; the self-test confirms that a tampered lattice ciphertext or a different context yields a different key.

4.4 Security tiers and the policy engine

Tier	KEM suite	Shares n / threshold t	Default trigger
T1 ephemeral	X25519 + ML-KEM-768	1 / 1	< 5 years, low sensitivity
T2 standard	X25519 + ML-KEM-768 + HQC-192	3 / 2	5-20 years, or score 35-64
T3 long-term	X25519 + ML-KEM-1024 + HQC-256	5 / 3	>= 20 years, or score >= 65

The policy engine is deliberately a transparent rule-based scorer, not a trained model. The exposure score (0-100) is the sum of: a lifetime term min(55, 14*log2(1+years)); filename rules (key or credential material +25, personal/financial/legal +18, backup/bulk +10); content rules on the first 256 KiB (PEM private key +30, AWS key +25, 'password =' +12, SSN-like +12, 12-digit ID-like +8; sensitivity capped at 45); and +5 for files over 1 GiB. Score below 35 gives T1, below 65 gives T2, otherwise T3. A lifetime floor (>= 20 years forces T3, >= 5 years forces T2) overrides a low score. Every contribution is printed so the decision is explainable. If the user picks a tier below the recommendation the tool prints a warning.

4.5 Package format (metadata privacy)



Figure 4.2: Layout of an encrypted package (.pq2pkg).

- The header only holds a random package id, cipher name, chunk size, chunk count and padded size.
- Every chunk is exactly 64 KiB (last one included). The plaintext is padded to a Padme size, so the ciphertext length reveals only a coarse size class.
- Each chunk's AAD binds the header hash, the chunk index and a 'last chunk' flag (STREAM construction), so reordering, truncation and extension fail authentication.
- The encrypted trailer contains the real file name, size, SHA3-256 hashes of ciphertext and plaintext, the secrecy lifetime, the signed minimum tier ('floor') and the sender's public bundle with a hybrid signature (Ed25519 and ML-DSA-65, both required).
- No recipient identifiers exist anywhere; a recipient finds its slots by trial decryption.

4.6 Protocols

Send

1. Load sender identity; run the policy engine to choose tier, suite, n and t; check the relay topology.
2. Generate package id and DEK; build the signed manifest; encrypt the file in a streaming pass.
3. Upload the ciphertext to relay 1 (all relays with --replicate); receive a signed log receipt.

4. Shamir-split the DEK; wrap each share for its holder with the hybrid KEM; distribute the slots round-robin over relays; collect receipts.
5. Write the ticket and the receipts file.

Receive

1. Download all slot bundles (each download is itself logged on the relay and returns a receipt).
2. Group bundles by epoch; for the newest epoch trial-decrypt all slots with the recipient's keys; add any releases from custodians.
3. If at least t shares are available, rebuild the DEK and authenticate it by decrypting the trailer (wrong share combinations fail the AEAD check).
4. Verify the sender signature, the descriptor MAC and the signed minimum-suite floor; check the epoch against the local rollback state.
5. Stream-decrypt every chunk, verify padding is zero, verify both hashes against the signed manifest, and only then move the file into place.

Threshold custody

With several --to recipients, each holds one share. A custodian runs `approve --for <combiner.pub>`, which re-encrypts that custodian's share to the combiner and signs it. The combiner supplies t-1 such release files together with their own slot to open the file.

Crypto-agility: audit and rewrap

`deprecate <ALG>` marks an algorithm as broken. `audit` fetches the newest epoch for each ticket and flags those that use a deprecated algorithm, fall below the tier required by their lifetime, or are older than a configured age. `audit --fix` and `rewrap` recover the DEK, choose a new suite (never below the sender's signed floor), and publish slot set epoch+1. The ciphertext is not touched. The receiver's state file remembers the highest epoch seen, so relays cannot silently serve an older, weaker epoch.

4.7 Relay channel and transparency log

A relay authenticates to the client by signing the handshake transcript with SLH-DSA (hash-based). Session keys come from ephemeral X25519 and ephemeral ML-KEM-768 secrets through HKDF, giving forward secrecy; records use AES-256-GCM with counter nonces. The client is anonymous. Relay keys are pinned on first use (or from the ticket). Each state-changing or key-fetching operation appends a record to the relay's log, a binary Merkle tree (SHA3-256, RFC 6962 domain separation). The relay returns a receipt: the leaf, the index, an inclusion proof, and a tree head signed with SLH-DSA, which can be checked offline. `monitor` stores the last tree head and requests a consistency proof to prove the new tree extends the old one.

4.8 Topology rule

If t or more shares were stored on one relay, that relay alone could rebuild the key (given broken KEMs). `send` therefore refuses to run if ceil(n / relays) >= t unless --allow-weak-topology is given, and states how many relays are needed.

5. Implementation

5.1 Technology stack

Component	Choice
Language	Python 3.9+ (single file, about 2,200 lines, one command-line entry point)
Classical crypto	cryptography library: AES-GCM, X25519, Ed25519, HKDF-SHA3-256, scrypt
Post-quantum crypto	liboqs via liboqs-python 0.16: ML-KEM-768/1024, HQC-192/256, ML-DSA-65, SLH-DSA-SHA2-128f/s
Networking	Plain TCP sockets, length-prefixed JSON, custom PQ-hybrid handshake, thread per connection
Storage	File system: pkgs/, slots/<pkg>/, log.jsonl (append-only), relay_id.json
Interface	argparse command-line with 15 sub-commands

5.2 Code organisation

Section of the program	Main functions / classes	Purpose
Algorithm registry	kem_generate/encaps/decaps, sig_generate/sign/verify, _pick	Uniform API over X25519, ML-KEM, HQC, Ed25519, ML-DSA, SLH-DSA; tolerates liboqs naming differences
Identities	new_identity, save_identity, load_identity	Bundle of all keys; secret half encrypted with scrypt + AES-GCM; fingerprint over the public bundle
Hybrid combiner	kem_combine, hybrid_encaps/decaps, hybrid_sign/verify	Multi-KEM key derivation, dual signatures
Secret sharing	shamir_split, shamir_combine, GF(256) tables	Threshold splitting with log/antilog arithmetic (AES polynomial 0x11B)
Padding	padme	Size hiding
Policy engine	assess, TIERS, effective_kems, suite_satisfies	Score, tier, suite, deprecation handling, floor comparison
Merkle log	mth, audit_path, verify_inclusion, consistency_proof, verify_consistency, MerkleLog	RFC 6962 tree with inclusion and consistency proofs
Channel	client_handshake, server_handshake, Channel	Authenticated forward-secret PQ-hybrid record layer
Relay	Relay (ops PKG_PUT, PKG_RANGE, SLOTS_PUT/GET, LOG_HEAD, LOG_CONS), RelayClient	Storage server and its client
Package	build_package, parse_pkg, open_trailer, verify_meta, decrypt_body	Streaming chunked AEAD container
Key wrap	wrap_slot, unwrap_slot, issue_slotset, make_release, recover, check_rollback	Slots, epochs, releases, recovery with all verifications
Commands + tests	cmd_* functions, cmd_selftest, cmd_demo	User interface, 24 unit checks, end-to-end demo

5.3 Notable implementation decisions

- **Fail closed.** recover() tries every combination of t shares and accepts one only if the trailer authenticates; then it additionally checks the sender's signed minimum tier and the descriptor MAC. Any failure raises an error instead of falling back to a weaker path.
- **Streaming.** Files are encrypted and decrypted chunk by chunk (decryption fetches 16 chunks per range request), so memory use does not grow with file size. Output is written to a .part file and moved into place only after all hashes match.
- **Upload safety on relays.** Package ids must match a 32-hex pattern (no path traversal); packages are immutable; uploads are validated (magic, header, minimum size) before being committed; incomplete uploads are deleted; there is a per-package bundle limit.
- **Output-path safety.** The sender-controlled file name is reduced to its base name and made unique, so a malicious sender cannot write outside the output directory.
- **Identity files.** Private keys are stored only in scrypt (N=2^15) + AES-GCM encrypted form with mode 0600.

5.4 Command-line interface

Command	Function
info / selftest / demo	Algorithm availability; 24 unit checks; full local end-to-end demo with three relays
keygen, trust	Create identity (.id private, .pub public); trust a sender fingerprint
assess	Dry-run of the policy engine with an explanation
relay	Run a relay and transparency-log node
send, receive	Encrypt and upload; download and decrypt
approve	Custodian releases its share to a combiner
deprecate, audit, rewrap	Mark algorithm as broken; find and fix outdated wraps
verify-receipt, monitor	Offline receipt verification; log append-only check

6. Testing, Results and Analysis

6.1 Test environment

Linux (cloud VM, single core used), Python 3, cryptography library, liboqs-python 0.16 (liboqs built from source). Relays and clients ran on localhost, so network latency is not included. Timings are means over repeated runs (20 runs for fast operations, 3 for the slow SLH-DSA signing) and are indicative; they will differ on another computer. All raw outputs are saved in the repository under results/.

6.2 Unit tests (selftest)

The command `python main.py selftest` ran 24 checks and all passed:

Group	Checks	Result
Shamir sharing	1-of-1, 2-of-3, 3-of-5, 7-of-7 reconstruct from both the first t and last t shares; t-1 shares give a wrong key	5 / 5 PASS
Padme	never shrinks, overhead at most 12.5% (+1 byte) on test lengths	1 / 1 PASS
Merkle tree	inclusion and consistency proofs for tree sizes 1, 2, 3, 5, 8, 13, 21, 50; rewritten history detected	9 / 9 PASS
Hybrid KEM	round trip for T2 and T3 suites; tampered lattice ciphertext changes key; different context changes key	6 / 6 PASS
Signatures	hybrid signature verifies and rejects another message; SLH-DSA sign/verify	3 / 3 PASS
Total		24 / 24 PASS

6.3 End-to-end scenarios (demo)

The command `python main.py demo` creates four identities, starts three relays on localhost and runs the following scenarios (total run time about 11 seconds, all assertions passed):

#	Scenario	Observed outcome
1	Policy dry run for 'wallet-backup.key' (1.3 MB, contains a PEM private key), 30 years	Score 100/100 with each reason printed (lifetime +55, key-material filename +25, backup filename +10, PEM private key +30); tier T3 selected (X25519 + ML-KEM-1024 + HQC-256, 3-of-5)
2	Send to Bob over 3 relays, then receive	Key recovered at epoch 1; sender verified; 20 chunks authenticated; output byte-for-byte identical to original
3	Verify receipts offline	4 / 4 receipts valid (inclusion proof + SLH-DSA tree-head signature)
4	Inspect what a relay stores	1,338,389-byte package for a 1,300,055-byte file (about 2.9% larger); file name and sender name not present in stored bytes
5	Flip one byte in the stored ciphertext	Rejected: 'chunk 11/20 failed authentication (tampered)'; no output file written
6	Send with a single relay and tier T2 (3 shares, t = 2)	Refused: topology too weak (3 shares on one relay, threshold 2)

#	Scenario	Observed outcome
7	Mark HQC-192 as broken; audit; audit --fix	Ticket using HQC-192 flagged NEEDS UPGRADE; rewrap created epoch 2 at tier T3; second audit reports 0 problems; ciphertext untouched
8	Relays hide the newer epoch (rollback / downgrade attack)	Rejected: 'ROLLBACK detected: you previously used epoch 2 ... relays now only offer epoch 1'
9	Threshold custody 2-of-3 (Bob, Carol, Dave)	Bob alone: 'have 1 share(s), need 2'. After Carol's signed release: file opened, approval from Carol reported
10	Log monitor	First observation recorded 18 entries; later 18 -> 19 entries verified as append-only via consistency proof
11	Policy warning	Forcing tier T2 when the engine recommends T3 printed a warning but was allowed
12	Repeat receive of an already downloaded file	Saved as wallet-backup_1.key (no overwrite)

6.4 Cryptographic primitive performance

Table 6.1: Key-encapsulation mechanisms (sizes in bytes, times in ms)

Algorithm	Family	Public key	Ciphertext	Keygen	Encaps	Decaps
X25519	elliptic curve	32	32	0.034	0.075	0.067
ML-KEM-768	lattice	1184	1088	0.019	0.018	0.022
ML-KEM-1024	lattice	1568	1568	0.020	0.022	0.025
HQC-192	code-based	4514	8978	4.00	7.28	13.5
HQC-256	code-based	7237	14421	10.5	20.3	41.0

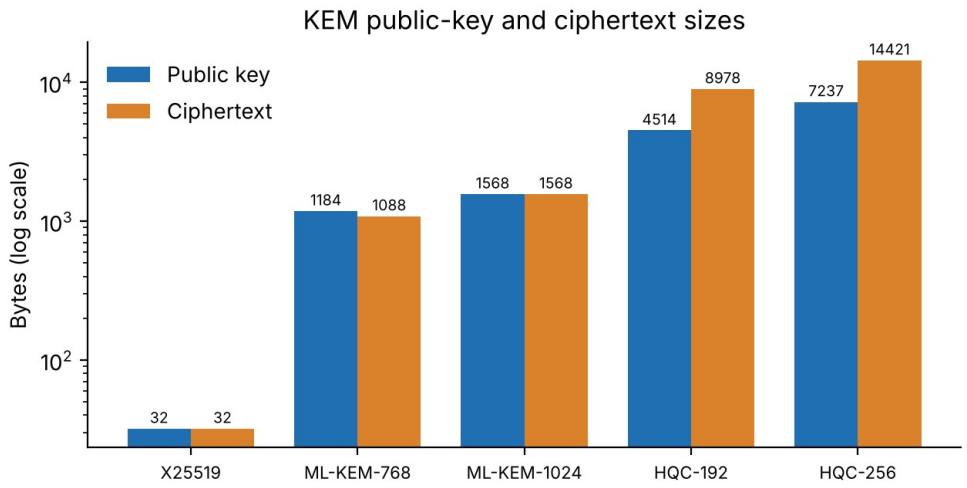


Figure 6.1: KEM public-key and ciphertext sizes.

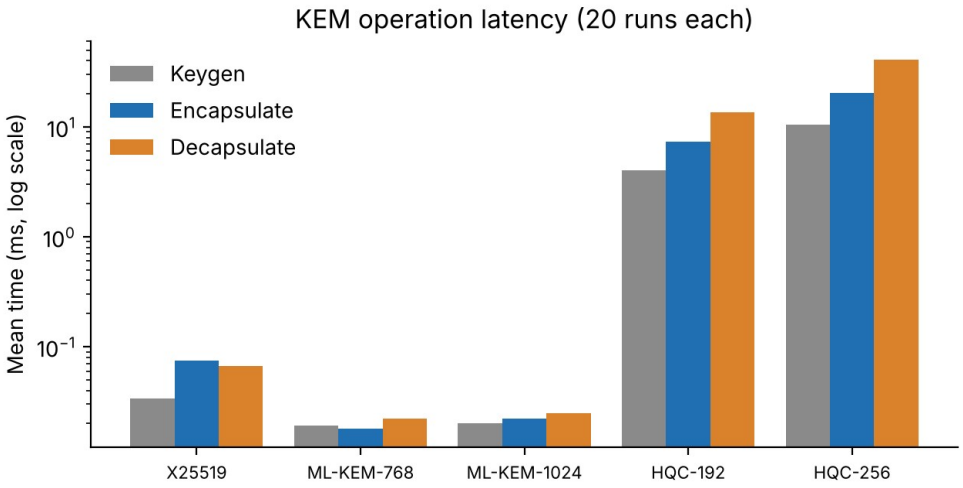


Figure 6.2: KEM operation latency (log scale).

Analysis. ML-KEM is the cheapest post-quantum choice: about 0.018 ms per operation with a 1.2 KB key and 1.1 KB ciphertext. HQC is roughly two orders of magnitude slower (decapsulation 13.5 ms for HQC-192 and 41.0 ms for HQC-256) and its ciphertexts are 9-14 KB. This is the price of code-based diversity; it is acceptable because the KEM is used only once per key share, not per byte of data.

Table 6.2: Signature algorithms

Algorithm	Used for	Public key (B)	Signature (B)	Sign (ms)	Verify (ms)
ed25519	manifest, release (classical half)	32	64	0.097	0.090
ML-DSA-65	manifest, release (PQ half)	1952	3309	0.128	0.052
SLH-DSA-SHA2-128f	relay identity, log heads, handshake	32	17088	31.2	2.22
SLH-DSA-SHA2-128s	alternate (smaller, slower)	32	7856	539	0.445

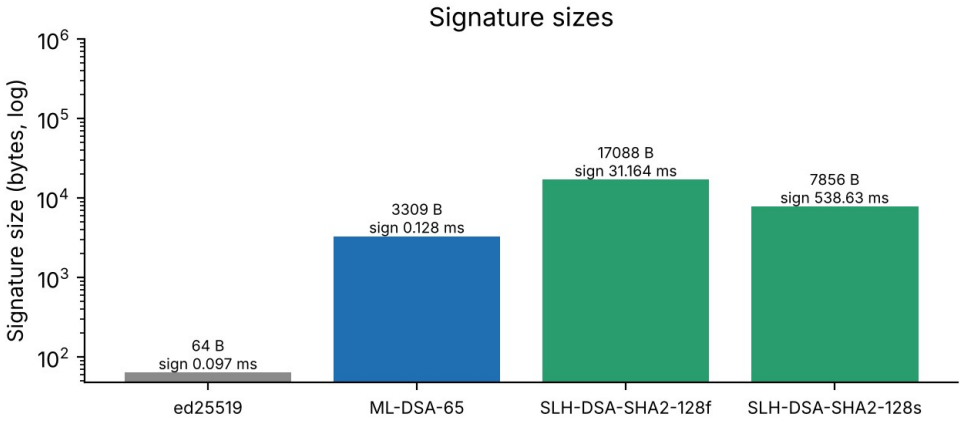


Figure 6.3: Signature sizes and signing times.

Analysis. The hash-based SLH-DSA signature is large (17088 bytes for the fast variant) and signs in about 31.2 ms, but it is only used by relays (once per handshake and per tree head), where its minimal assumption (only a secure hash function) is the point. The 's' variant halves the size (7856 bytes) but

needs about 539 ms to sign, which would make every receipt noticeably slower; the program prefers the 'f' variant. Verification stays fast (2.22 ms), which matters for clients.

6.5 Cost of each security tier

Table 6.3: Hybrid KEM cost per key share

Tier	Suite	n / t	Ciphertext bytes	Encaps (ms)	Decaps (ms)
T1	X25519 + ML-KEM-768	1 / 1	1120	0.104	0.101
T2	X25519 + ML-KEM-768 + HQC-192	3 / 2	10098	9.65	14.8
T3	X25519 + ML-KEM-1024 + HQC-256	5 / 3	16021	23.1	35.3

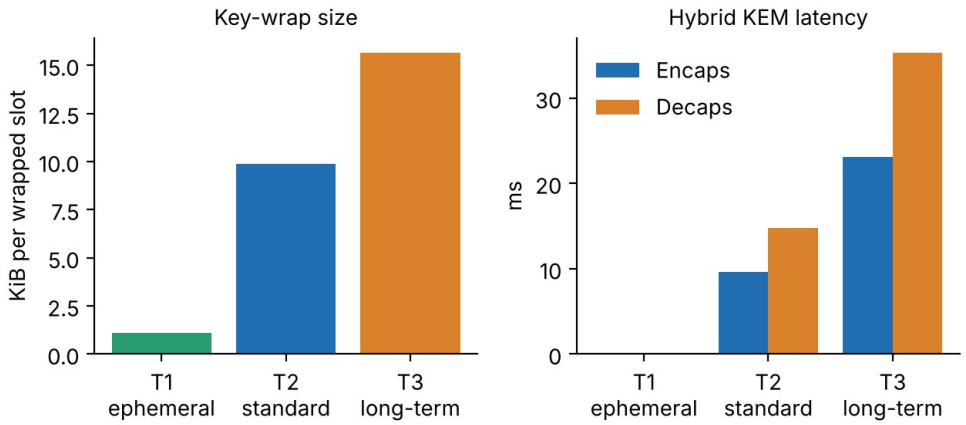


Figure 6.4: Size and latency of the three tiers.

Analysis. Moving from T1 to T3 grows the wrapped-key size from about 1.1 KiB to 15.6 KiB per share and the latency from about 0.104 ms to 23.1 ms. With five shares a T3 send spends about 115 ms on key wrapping and a T3 receive about 106 ms for three unwraps, plus trial attempts on slots not meant for the recipient. A full serialized T3 slot is about 21.0 KiB of JSON. The tier cost is negligible compared with network transfer time for any realistic file, which supports defaulting long-lived files to the strongest tier.

6.6 File encryption throughput and storage overhead

Table 6.4: Local streaming encryption / decryption (AES-256-GCM, 64 KiB chunks, tier T2)

File size	Package size	Overhead	Encrypt (s)	Decrypt + verify (s)	Enc MiB/s	Dec MiB/s
0.01 MiB	38093	263.31 %	0.001	0	9.80	85.0
0.10 MiB	133855	27.65 %	0.002	0.001	53.7	160
1.00 MiB	1076162	2.63 %	0.006	0.007	155	148
10.00 MiB	10515653	0.29 %	0.056	0.065	178	153
50.00 MiB	52468933	0.08 %	0.268	0.334	186	150

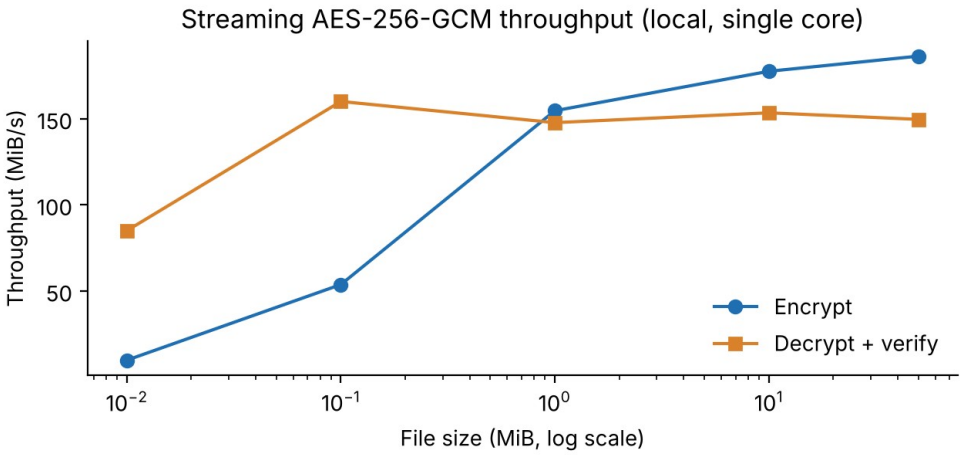


Figure 6.5: Throughput versus file size.

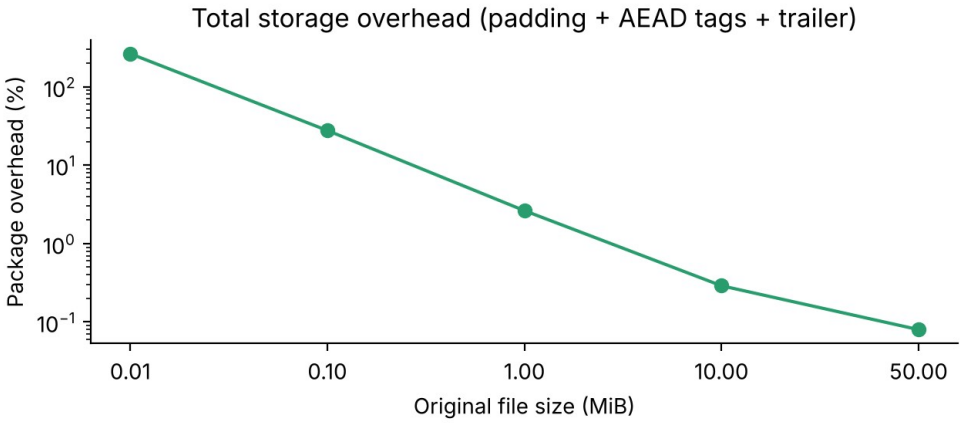


Figure 6.6: Package overhead versus file size.

Analysis. Throughput stabilises around 178-186 MiB/s for encryption and about 150 MiB/s for decryption with full verification (hash of ciphertext and plaintext, per-chunk authentication, padding check) on one core, so the symmetric layer will not be the bottleneck on typical networks. Small files show low throughput because fixed costs (writing the trailer, signing the manifest with Ed25519 and ML-DSA) dominate. Storage overhead is large only for tiny files: a 10 KB file becomes about 38 KB because the encrypted trailer carries the sender's complete public key bundle (about 27 KB, mostly HQC and ML-KEM keys) and small files are padded to at least 4 KiB. For 1 MiB the overhead is 2.63% and for 10 MiB and above it is below 0.3%. A possible optimisation (future work) is to reference the sender by fingerprint instead of embedding the full bundle when the recipient already has it.

6.7 Padme padding

Table 6.5: Padme examples

Real length (B)	Padded length (B)	Overhead
4097	4352	6.22 %
10000	10240	2.4 %
100000	100352	0.35 %
1300055	1310720	0.82 %
10000000	10223616	2.24 %

Real length (B)	Padded length (B)	Overhead
100000000	100663296	0.66 %

The measured overhead is below 6.3% on all samples and always within the theoretical 12% bound. Files whose lengths fall into the same padded size are indistinguishable by size, which limits what a relay can infer about content type.

6.8 Secret sharing and transparency log cost

Shamir t-of-n	Split (ms)	Combine (ms)
2-of-3	0.026	0.012
3-of-5	0.047	0.016
6-of-10	0.163	0.029
16-of-32	1.366	0.121

Log entries	Inclusion proof length	Proof size (bytes)	Root computation (ms)
16	4	128	0.017
256	8	256	0.335
4096	12	384	5.09

Shamir operations on a 32-byte key take well under 2 ms even for 16-of-32. Inclusion proofs grow logarithmically: 12 hashes (384 bytes) for 4,096 log entries. Recomputing the full root is linear in the log size (5 ms for 4,096 leaves); the prototype recomputes the tree for each head, which would need an incremental tree for very large logs.

6.9 Summary of findings

- All 24 unit tests and all 12 demo scenarios behaved as designed, including every negative (attack) scenario.
- A single flipped ciphertext byte is detected before any output is released.
- Rollback, weak topology, insufficient custodians and unknown senders are all refused rather than silently tolerated.
- Upgrading an algorithm needs only a re-issue of a few KB of key material; the file is not re-encrypted.
- The post-quantum overhead is concentrated in the key wrap (ms, KB) and is irrelevant for bulk data, where AES-GCM speed dominates.
- Highest costs: HQC (slow, large ciphertext) and tiny-file overhead from embedded public keys.

7. Security Analysis and Limitations

7.1 Properties and why they hold

Property	Argument
Confidentiality against one broken family	Wrap key = HKDF of all component secrets. Breaking only lattice or only code or only elliptic curve leaves the other secrets unknown.
Confidentiality against fewer than t relays	Shamir over GF(256): t-1 shares are statistically independent of the secret; holds even against unbounded computation. The topology rule prevents a single relay from holding t shares by mistake.
Integrity and ordering	Per-chunk AES-GCM with index and last-flag in AAD; ciphertext and plaintext hashes in the signed manifest; trailer bound to header hash.
Sender authenticity	Manifest is signed with Ed25519 and ML-DSA-65; both must verify. Sender fingerprint is checked against the local contact list.
Downgrade and rollback resistance	Epoch and suite are part of the KEM context; descriptor MAC under the DEK; sender-signed minimum suite; local highest-epoch state.
Relay accountability	Receipts with inclusion proofs signed by a hash-based signature; consistency proofs expose rewriting of history (assuming a monitor sees a consistent view).
Metadata privacy	Name, size, hash, sender in AEAD trailer; Padme; equal chunks; anonymous client; trial decryption instead of recipient ids.

7.2 Honest limitations

- 'Quantum-proof' means no known quantum attack. Nothing is proven; that is the reason for the hybrid design.
- liboqs describes itself as research-grade, HQC's final standard may still be pending, and the program has not been audited or tested against side channels.
- Rewrap protects future recordings only: slots already harvested under an old suite cannot be recalled.
- Relays still see IP addresses, access times and ciphertext sizes (coarsened). t colluding relays combined with a break of all KEM families in the suite would recover the key.
- Anonymous uploads mean a public relay could be filled with data; the prototype has size and bundle-count limits but no rate limiting or authentication.
- The relay's SLH-DSA private key is stored unencrypted on disk (file mode 0600). Identity files are only as strong as their passphrase.
- Trust on first use for relay keys and for contacts requires out-of-band fingerprint comparison.
- The policy engine is a keyword and pattern heuristic; it can be fooled by misleading names and does not understand file content beyond a few patterns.
- The Merkle monitor needs to be run regularly and against several observers to catch a relay that shows different histories to different people (split-view attack).

8. Conclusion and Future Work

PQFS 2.0 meets all seven objectives. It demonstrates that post-quantum readiness is more than swapping one algorithm: the design combines three independent mathematical families, information-theoretic key splitting across servers, a safe upgrade path, an explainable policy for choosing strength, and verifiable relay behaviour, while keeping bulk-data performance at AES speed. The test suite and the demonstration show that both normal operation and the modelled attacks behave as intended.

Future work

- Reference sender by fingerprint to cut the fixed overhead on small files.
- Incremental Merkle tree and gossip between independent monitors to defeat split-view attacks.
- Authenticated uploads, rate limiting and quota per sender; encrypted storage of the relay signing key.
- Resumable transfers and parallel multi-relay download; a GUI or web front-end.
- Constant-time review of the Python-side code paths and fuzzing of the relay protocol parser.
- Final HQC standard parameters when published; ML-KEM-1024 and further algorithms added to the tier table without changing the file format.

References

[1] NIST, FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM), 2024.

[2] NIST, FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA), 2024.

[3] NIST, FIPS 205: Stateless Hash-Based Digital Signature Standard (SLH-DSA), 2024.

[4] NIST, Status report on the fourth round of the NIST PQC standardization process; selection of HQC, 2025.

[5] P. Shor, Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer, SIAM J. Computing, 1997.

[6] A. Shamir, How to share a secret, Communications of the ACM 22(11), 1979.

[7] B. Laurie, A. Langley, E. Kasper, RFC 6962: Certificate Transparency, 2013; and RFC 9162 (v2), 2021.

[8] A. Nikitin, et al., Reducing metadata leakage from encrypted files and communication with PURBs, Proc. Privacy Enhancing Technologies 2019 (Padme).

[9] M. Dworkin, NIST SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM), 2007; H. Krawczyk, P. Eronen, RFC 5869: HKDF, 2010.

[10] A. Langley, M. Hamburg, S. Turner, RFC 7748: Elliptic Curves for Security (X25519), 2016; S. Josefsson, I. Liusvaara, RFC 8032 (Ed25519), 2017.

[11] Open Quantum Safe project, liboqs and liboqs-python, https://openquantumsafe.org.

[12] pyca/cryptography documentation, https://cryptography.io.

Appendix A: How to run the prototype

Source code: https://github.com/theveryholypenguin/pqfs2

```
pip install cryptography liboqs-python
python main.py info        # check algorithm availability
python main.py selftest    # 24 unit checks
python main.py demo        # complete end-to-end demo on localhost
```

Manual use with three relays (three terminals) and two identities:

```
python main.py keygen alice ; python main.py keygen bob
python main.py relay --port 5001 --store ./r1   (repeat for 5002 / ./r2 and 5003 /
./r3)
python main.py send report.pdf --id alice.id --to bob.pub --relay 127.0.0.1:5001
--relay 127.0.0.1:5002 --relay 127.0.0.1:5003 --years 30
python main.py trust alice.pub          (run by Bob)
python main.py receive <pkg>.ticket.json --id bob.id --outdir ./inbox
```

Appendix B: Mapping to evaluation rubrics

Criterion (marks)	Where addressed
Problem Statement & Design (2)	Chapters 2, 3 and 4; Figures 4.1, 4.2
Implementation & Technical Accuracy (2)	Chapter 5; Chapter 7; source file main.py with 24 self-tests
Result Analysis (2)	Chapter 6: Tables 6.1-6.5, Figures 6.1-6.6, summary 6.9
Demonstration & Viva (2)	demo command; separate demo script and viva preparation sheet
Documentation & Timely Submission (1)	This report; README.md in the repository
GitHub upload & LinkedIn poster post (1)	https://github.com/theveryholypenguin/pqfs2 and the LinkedIn post